OTUS

Kotlin Backend Developer.Professional

otus.ru

REC

Проверить, идет ли запись

Меня хорошо видно && слышно?

Ставьте “+” в чат, если да
Напишите, если есть проблемы

Правила вебинара

Активно
участвуем

Off-topic обсуждаем в
OTUS-Kotlin-2025-05

Задаем вопрос
в чат или голосом

Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения

Индивидуально

Время, необходимое
на активность

Пишем в чат

Говорим голосом

Документ

Ответьте себе или
задайте вопрос

Объектно-ориентированное программирование

Модуль 1: Введение в Kotlin	Введение	Базовые элементы	Функц. элементы	ООП		
Модуль 2: Расширенные возможности Kotlin	DSL	Корутины	Sequence, Flow	Multi-platform	Interop	Gradle
Модуль 3: Подготовка к разработке	Выбор проекта	Маркетинг аналитика	Security	Архитек-тура	DevOps	
Модуль 4: Разработка API	OpenAPI	Модели и маперы	Testing	Monitoring		
Модуль 5: Фреймворки и асинхронный транспорт	Spring	Ktor	WebSocket	RabbitMQ	Kafka	Serverless
Модуль 6: Бизнес-логика проекта	Основы БЛ	Модуль БЛ	Валидация			
Модуль 7: Работа с базами данных	Работа с БД	Репози-торий	Конкурен-ция	Postgres	Cassandra	Gremlin
Модуль 8: Развитие бизнес-логики проекта	Машина состояний	Auth	Основы ML	ML в бэкенде	Обзор экосист.	
Модуль 9: Проектная работа	Защита					

Маршрут вебинара

- ООП - введение
- Элементы ООП
- Принципы ООП
- Обобщенное программирование
- Специальные классы
- Дополнительные возможности
- Рефлексия

Цели вебинара

После занятия вы:

1. Будете знать элементы объектной модели Kotlin
2. Узнаете, как создавать различные классы и объекты
3. Ответите на вопрос, как выполнять наследование и делегирование
4. Узнаете, как пользоваться геттерами и сеттерами

Смысл

Зачем вам это уметь

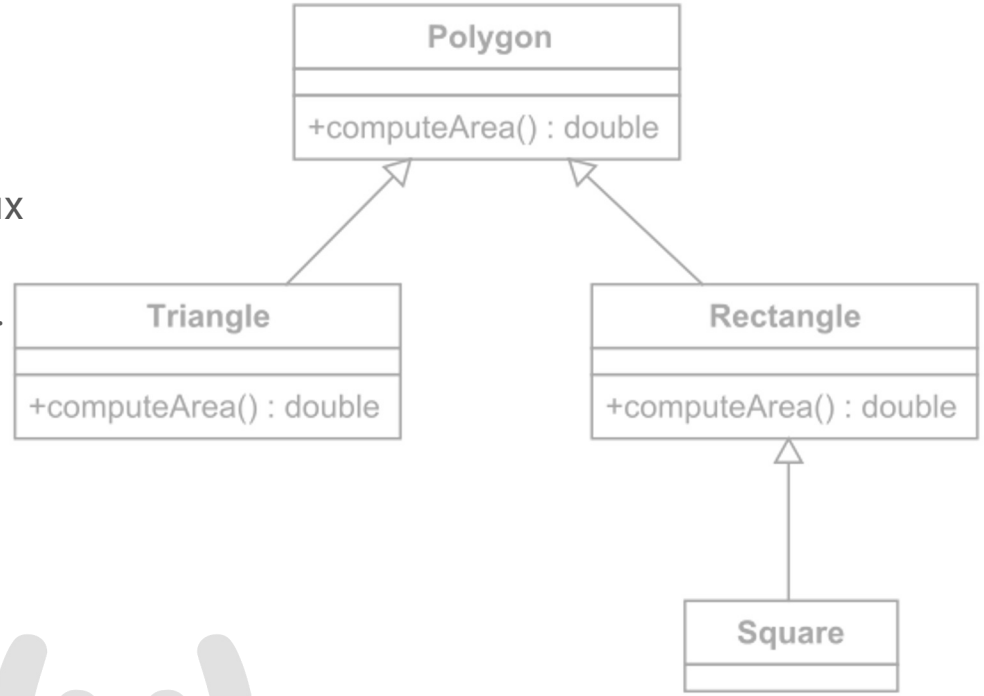
1. Чтобы эффективно разрабатывать и поддерживать ПО на языке Kotlin
2. Чтобы уметь обосновать выбор того или иного решения в контексте дизайна ПО

ООП - введение

Дисклеймер

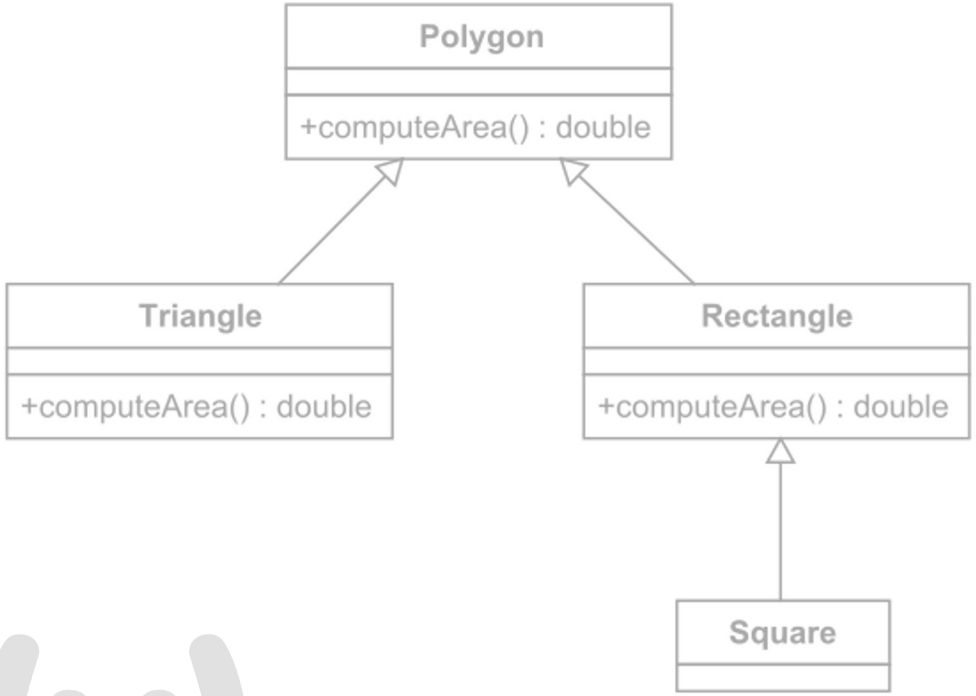
Эта лекция

- Не научит вас ООП с нуля
- Не даст глубины, если вы уже имеете большой опыт в Java
- Нужно рассматривать её как overview о нюансах ООП в Kotlin, важных для дальнейшего хода курса.
- Приобретать опыт будем в течении последующих месяцев обучения.



Много парадигм: ООП – не единственная

- Декларативное
  - Что должно получиться
- Процедурное (императивное) программирование
  - Разбивка на процедуры и функции, запрет goto
- Функциональное программирование
  - Возврат результатов функций, запрет состояний
- Объектно-ориентированное программирование
  - связь структур и функций
- Делегирование
  - запрет на бесконтрольное наследование
  - борьба с сильным связыванием



ООП — определение и основная концепция

- Методология программирования, основанная на представлении программы в виде совокупности взаимодействующих объектов, каждый из которых является экземпляром определенного класса, а классы образуют иерархию наследования
- Решает задачу структурирования кода с точки зрения управляемости
- Транслирует понимание задачи программистом в наиболее удобную для дальнейшего использования форму.

Ключевые понятия ООП

- Интерфейс

  Контракт, определяющий множество операций, которые должен поддерживать объект

  пример: Comparable<T> ~ [int comparable(T other)]

- Тип данных

  множество допустимых значений и операций над ними

  *пример: int ~ [-2 147 483 648 .. 2 147 483 647], [+, -, *, /, %]*

- Класс

  Комплексный тип данных, состоящий из единого набора «полей» (переменных элементарных типов) и «методов» (функций для работы с этими полями) для всех объектов этого класса

  пример: Cash – комплексный тип для представления денежных средств

- Объект

  Сущность, появляющаяся в программе при создании экземпляра класса

  пример: экземпляры класса Cash income и expense

Элементы ООП

Интерфейсы

- определяет свойства и методы, реализацию которых обязан обеспечить наследующий его класс, то есть контракт
- классы и интерфейсы могут наследовать несколько интерфейсов

Интерфейсы

- определяет свойства и методы, реализацию которых обязан обеспечить реализующий его класс, то есть контракт
- классы и интерфейсы могут наследовать несколько интерфейсов

свойство

имя интерфейса

```
interface Sum {
    val result: Long
    fun add(summand: Long)
    fun addAll(summands: Iterable<Long>) {
     summands.forEach { add(it) }
    }
```

метод

Метод с реализацией по умолчанию

```
class DefaultSum : Sum {
    override var result: Long = 0L
      private set

    override fun add(summand: Long) {
        result += summand
    }
}
```

Класс, реализующий интерфейс Sum

Классы

- основной «строительный материал» программы в ООП-стиле

Классы

- основной «строительный материал» программы в ООП-стиле

```
class Cash  constructor (
    val amount: BigDecimal,
    val currency: Currency,
    argument: String
) {
    constructor (
        amount: String,  currency: Currency
    ) : this(BigDecimal(amount), currency, "Arg")

    fun format(locale: Locale): String {
        val formatter = NumberFormat.getCurrencyInstance(locale)
        formatter.currency = currency
        return formatter.format(amount)
    }

    operator fun minus(other: Cash): Cash {
        require(currency == other.currency) {
            "Summand should be of the same currency"
        } // throws an IllegalArgumentException in case result is false
        return Cash(amount - other.amount, currency)
    }
}
```

Первичный конструктор

свойства

Имя класса

вторичный конструктор

оператор

метод

Конструкторы

- Обеспечивают инициализацию экземпляров класса
- Первичный конструктор – часть заголовка класса, если отсутствуют какие-либо модификаторы или аннотации, можно опустить ключевое слово “constructor”. Также в первичном конструкторе можно декларировать свойства, используя ключевые слова var/val, при этом код инициализации располагают в init-блоке (:

```
class Cash constructor(val amount: BigDecimal, val currency: Currency, arg: String)
```

- Вторичный конструктор: обязательно использовать ключевое слово constructor, также необходимо делегировать вызов первичного конструктора, используя ключевое слово this:

```
constructor(
        amount: String, currency: Currency
    ) : this(BigDecimal(amount), currency, "arg")
```

- Создание объектов через вызов конструктора (без new!):

```
val income = Cash(BigDecimal("100.50"), usd, "arg")
```

Абстрактный класс

- базовый класс не предполагающий создание экземпляра, при этом часть функциональности уже может быть реализована
- может содержать абстрактные методы и свойства (не имеющие реализации)
- абстрактный класс и абстрактные члены open по умолчанию

```
abstract class AbstractBase{
   abstract val abstractProperty: String
   abstract fun abstractMethod(): Double

   val property: String = ""
    fun method() {}
}
```

Объекты

- Два способа создания объектов:

Вызов конструктора

  - Как экземпляр класса через конструктор

```
val income = Cash(BigDecimal("100.50"), usd)
```

  - Используя объектный литерал

Создание объекта с именем GlobalIdGenerator

```
object GlobalIdGenerator: Iterator<Long> {
    private var nextId: Long = 0L
    override fun hasNext(): Boolean = true
    override fun next(): Long = nextId++
}
```

Анонимный объект aka object expression

```
someFun(object : Iterator<Long> {
    private var nextId: Long = 0L
    override fun hasNext(): Boolean = true
    override fun next(): Long = nextId++
})
```

companion

Позволяет делать некоторые члены класса быть общими для всех объектов

- Можно объявить объект-singleton внутри класса, используя ключевое слово companion
- Использовать члены companion-объекта можно, используя имя класса в качестве квалификатора

```
class SomeClass {
   companion object {
      const val PI = 3.1415
      fun build() = SomeClass()
   }
}

println(SomeClass.PI)
println(SomeClass.Companion.PI)
var a = SomeClass.build()
```

Создание и использование объектов

```
// Создание объекта (первичный конструктор)
val income = Cash(BigDecimal("100.50"), usd)

// Создание объекта (вторичный конструктор)
val expense = Cash("50.40", usd)

// Вызов оператора minus
val profit = income - expense

// Вызов метода
val formatted = profit.format(Locale.US)

// Чтение свойства
val amount= profit.amount

// Запись свойства (не сработает - у нас val)
profit.amount = amount
```

Свойства

- Обеспечивают хранение и доступ к данным объекта
- Могут быть объявлены изменяемыми (**var**) и неизменяемыми (**val**)
- Методы доступа **get** и **set** (set только для изменяемых свойств), опциональны
- Автоматически создаются backing fields

```
// В первичном конструкторе и теле
класса
val immutableProperty: String = ""
var mutableProperty: String = ""

// В теле класса
val immutableProperty: String = ""
   get() { return field }

var mutableProperty: String = ""
   get() { return field }
   set(value) { field = value }
```

```
// В первичном конструкторе и теле класса

val <propertyName>[: PropertyType][=
        property_initializer>]  [<getter>]
        [<setter>]
```

Класс Any

Базовый класс, от которого наследуются все остальные

Функции Any:

- `fun toString(): String`
- `fun hashCode(): Int`
- `fun equals(other: Any?): Boolean`

Any: toString()

```
Строковое представление объекта

fun toString(): String
```

- ru.otus.otuskotlin.oop.Cash@7ef20235 - вот что вернется по умолчанию, поэтому желательно чтобы возвращала что-то человекочитаемое
- **Примеры неявных вызовов toString():**
  - println(x)
  - println(listOf(x, y))
  - var a = “Hello, ” + world
  - var a = “Hello, $world”
  - В отладчике

Any: equals

Идентичность и равенство.

- переменные идентичны, когда ссылаются на один объект
- значение переменных равно, когда **equals** возвращает *true*

```
val a = BigDecimal("10")
val b = BigDecimal("10")

assertTrue(a == b)
assertFalse(a === b)
```

Требования к equals:

- **Рефлексивный** (Reflexive): for any non-null value x, x.equals(x) should return true.
- **Симметричный** (Symmetric): for any non-null values x and y, x.equals(y) should return true if and only if y.equals(x) returns true.
- **Транзитивный** (Transitive): for any non-null values x, y, and z, if x.equals(y) returns true and y.equals(z) returns true, then x.equals(z) should return true.
- **Консистентный** (Consistent): for any non-null values x and y, multiple invocations of x.equals(y) consistently return true or consistently return false, provided no information used in equals comparisons on the objects is modified.
- Never equal to null: for any non-null value x, x.equals(null) should return false.

Any: hashCode

- Генерирует **hashCode** объекта (одинаковый для равных объектов)

```
val a = BigDecimal("10")
val b = BigDecimal("10")

assertTrue(a.hashCode == b.hashCode())
assertTrue(a == b)
```

Требования к hashCode:

- Consistent
- If two objects are equal according to the equals() method, then calling the hashCode method on each of the two objects must produce the same integer result.

LIVE

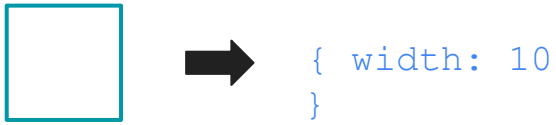
ex0-ex2

Вопросы?

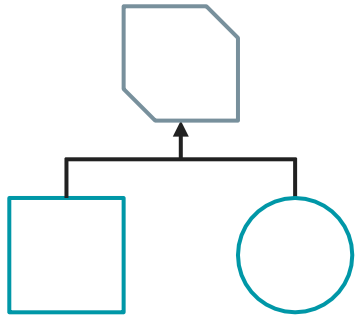
Основные элементы ООП?

Принципы ООП

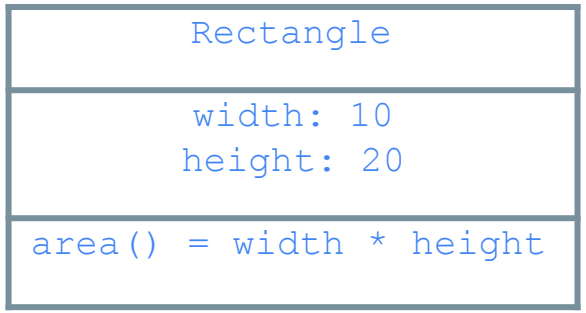
Ключевые принципы



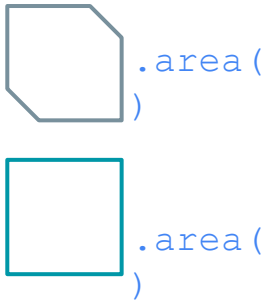
Абстракция данных



Наследование



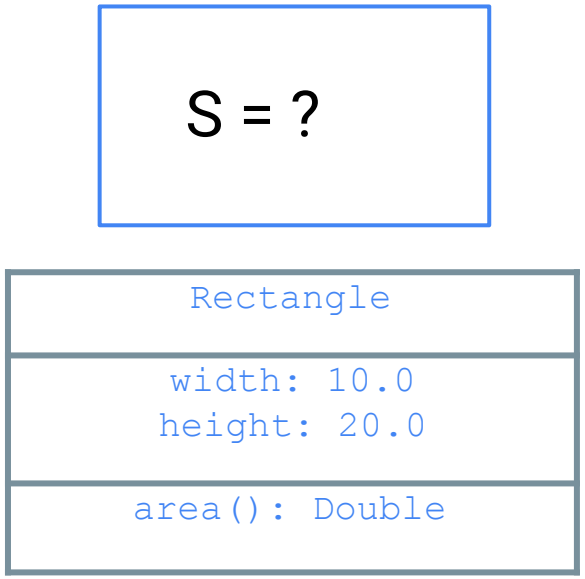
Инкапсуляция

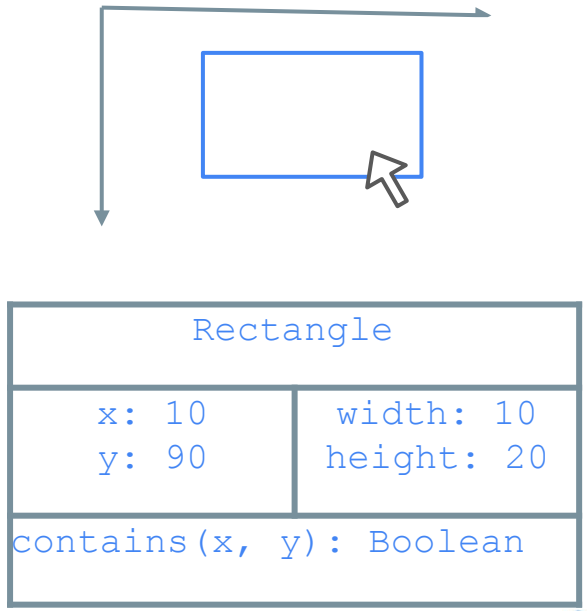


Полиморфизм

Абстракция данных

- Использование только тех характеристик объекта, которые с достаточной точностью представляют его в данной системе
- Основная идея: представить объект минимальным набором полей и методов и при этом с достаточной точностью для решаемой задачи





Инкапсуляция

- объединение данных и операций над ними в одном компоненте — **классе**
- предназначена для изоляции контрактных обязательств абстракции (протокол/интерфейс) от их реализации

Kotlin

```
class Rectangle(
   val width: Double,
   val height: Double,
) {
   fun area(): Double = width * height
}
```

Сокрытие

- разграничение доступа различных частей программы к внутренним компонентам друг друга

Элемент класса	Пример
`visibility` constructor(...)	`private constructor` (currency: Currency)
`visibility` val property: String	`protected val` property: String
`visibility` var property: String [`visibility` set]	`internal var` property: String private set
`visibility` fun method(...) {...}	`public fun` method() {...}
`visibility` operator invoke(...) {...}	`internal operator fun` invoke(): String = ...

Контроль видимости членов класса

Модификатор	Доступность
public	Доступен отовсюду. Применяется по умолчанию
internal	Доступен отовсюду в пределах модуля (Intellij module, Maven project, Gradle source set)
protected	Доступен в методах класса и его потомков
private	Доступен в методах класса

Наследование

- позволяет описать новый компонент на основе уже существующего с частично или полностью заимствованной функциональностью
- **дочерний класс** наследует свойства и поля **базового класса**

```
open class Rectangle(
    val width: Double,
    val height: Double
) {
   fun area(): Double = width * height
}

val rectangle = Rectangle(10.0, 20.0)
val area = rectangle.area()
```

```
class ScalableRectangle(  width: Double,
   height: Double,
) : Rectangle(width, height) {
     fun scaled(factor: Double) = ScalableRectangle(
        width * factor,
        height * factor
    )
}

val rectangle = ScalableRectangle(10.0, 20.0)
val area = rectangle.scaled(2.0).area()
```

Контроль над наследованием

Модификатор класса	Описание
final	Наследование запрещено. Применяется по умолчанию
sealed	Закрытое множество подтипов. Все потомки заранее известны компилятору
open	Открытое множество подтипов. Число потомков неограниченно и заранее не известно компилятору

Контроль переопределения

Модификатор	Описание
final	Запрещает переопределять свойства или методы. Применяется по умолчанию
open	Позволяет переопределять свойства или методы

Полиморфизм

- Принципиальная возможность для одного и того же кода обрабатывать данные разных типов
- Разновидности полиморфизма:
  - Универсальный
    - Параметрический (реализован в контексте обобщенного программирования, генерики)
    - Подтипов (наследование)
  - Ad-hoc (статический)
    - Приведение типов
    - Перегрузка

Обобщенное программирование

Параметрический полиморфизм

- описание данных и алгоритмов, которое можно применять к различным типам данных, не меняя само это описание
- Тема, достойная отдельного вебинара – можно говорить об инвариантности, ковариантности и контрвариантности в контексте обобщенного программирования

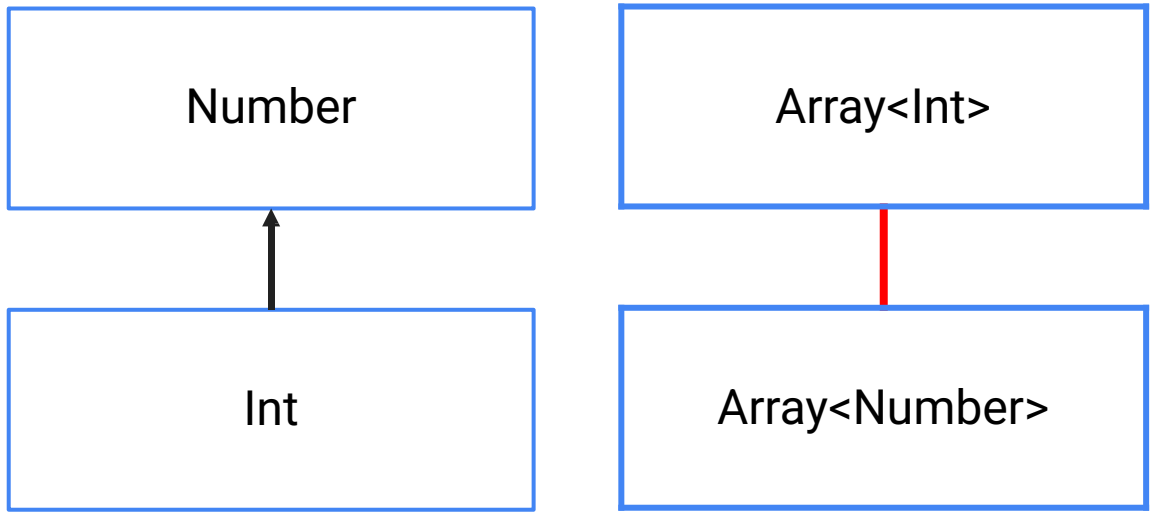
```
class Array<T>(size: Int) {
   operator fun get(index: Int): T = …
   operator fun set(index: Int, value: T) = ...
}

val intArray = Array<Int>(2)
intArray[0] = 100
intArray[1] = 500

val strArray = Array<String>(2)
strArray[0] = "foo"
strArray[1] = "bar"

// Вызовет ошибку компиляции
val objArray: Array<Any> = intArray
```

Инвариантность




```
val ints: Array<Int> = ...
val numbers: Array<Number> = ints // ERROR

// гипотетически
numbers[0] = 2.0
val someInt: Int = ints[0] // OOPS
```

Ковариантность, контравариантность

Инвариантность

```
interface Producer<T> {
    fun produce(t: T): T
}
```

T принимается и возвращается в/из функции, но не наследуется, т.е. фиксирована

Ковариантность

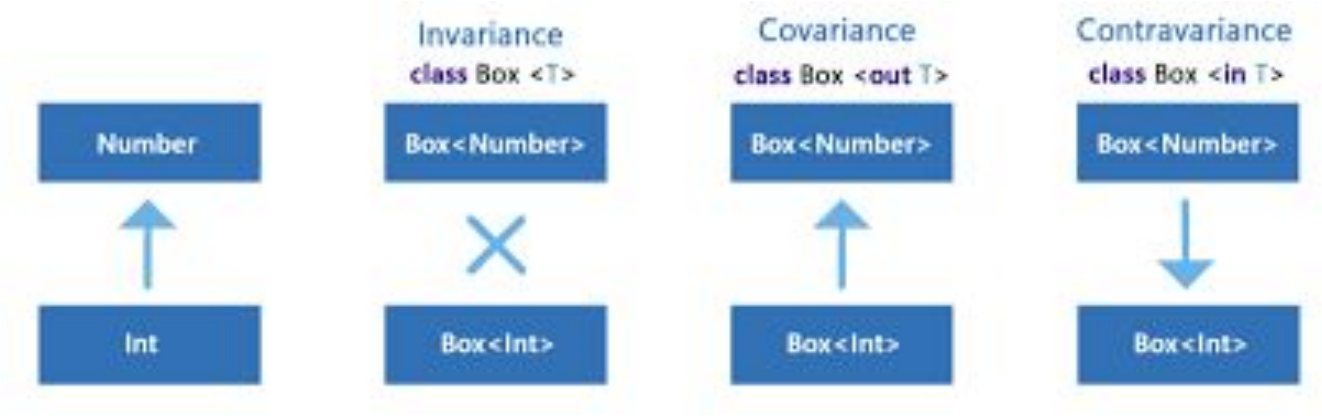
```
interface Producer<out T> {
    fun produce(): T
}
```

T только возвращается (produce) из функции

Контравариантность

```
interface Consumer<in T> {
    fun consume(t: T)
}
```

T только читается (consume) в функции



Вопросы?

Специальные классы

enum-класс

- типобезопасное перечисление
- позволяет добавлять свойства и методы
- не может наследовать другой класс, но может реализовать интерфейсы

```
enum class Day {
    Monday,
    Tuesday,
    Wednesday,
    Thursday,
    Friday,
    Saturday,
    Sunday,
}
```

```
fun doOnDay(day: Day) {
 when(day) {
     Day.Monday -> // Do something,
     Day.Tuesday, Day.Wednesday -> // Do other stuff
     else -> // some default thing to do
   }
}
```

data-класс

- предназначен для хранения данных (Data Transfer Object)
- **final**
- **equals, hashCode, toString** реализованы на основе свойств
- **copy** – метод для копирования с предопределенными значениями свойств

```
val product1 = Product("t-shirt", 20.0)

val product2 = Product("t-shirt", 20.0)


data class Product(
    val name: String,
    val price: Double,
    val inStock: Boolean = true
)
```

```
assertEquals(product1, product2)
assertEquals(product1.hashCode(), product2.hashCode())

println(product1.toString())
// out: Product(name=t-shirt, price=20.0, inStock=true)

println(product1.copy(inStock = false).toString())
// out: Product(name=t-shirt, price=20.0, inStock=false)
```

sealed-класс

Позволяет обеспечить закрытое множество подтипов. Все потомки заранее известны компилятору.

```
fun log(e: Error) = when(e) {
        is FileReadError -> { println("Error while reading file ${e.file}") }
        is DatabaseError -> { println("Error while reading from database${e.source}")}  is
RuntimeError -> { println("Runtime error") }
        // the `else` clause is not required because all the cases are covered
}
```

Inline/value-класс

- обертка над типом, при этом должны иметь единственное свойство, инициализируемое в основном конструкторе
- Основная идея – конкретизировать контекст использвольвания того или иного типа, не теряя производительность примитивных типов
- позволяет добавлять свойства (динамические) и методы
- оптимизирует производительность используя boxing/unboxing
- не может наследовать другой класс, но может реализовать интерфейсы

```
value class ProductId(
    val value: String
)

// Since kotlin 1.5.0
@JvmInline
value class ProductId(
    val value: String
)
```

```
val id1 = ProductId("t-shirt")
val id2 = ProductId("t-shirt")

assertEquals(id1, id2)
assertEquals(id1.hashCode(), id2.hashCode())

println(id1.toString())
// out: ProductId(value=t-shirt)
```

Вопросы?

Дополнительные возможности

lateinit

- Добавлен создателями языка для поддержки ситуаций, когда свойство инициализируется через Dependency Injection и подобных случаев
- Существует, но рекомендуется ограничить использование – он опасен, ведь в случае, если свойство не инициализировано, мы можем получить ошибку при доступе!

```
class MyTest {
    private lateinit var subject: TestSubject

    @Before
    fun setup(): Unit {
        subject = TestSubject()
    }

    @Test
    fun test() {
        if (!this::subject.isInitialized) fail("subject variable is not initialized")

        subject.method()
    }
}
```

Делегирование свойств

- Повторное использование общего поведения различных свойств – к примеру, ленивая инициализация, вызов кода при изменении свойств или же хранение данных в Map вместо отдельных полей
- Используете ключевое слов **by** => получаете delegated properties
- вызовы **get** делегируется оператору **getValue** объекта-делеагата
- вызов **set** делегируется оператору **setValue** объекта-делегата

```
class MappedState(
    state: MutableMap<String, String>
) {
    val id by state
    var name by state
    var description by state
}
class RandomInt {
    val value by lazy { Random.nextInt() }
}
```

Делегирование интерфейсов

- позволяет доверить обработку свойств и методов интерфейса объекту реализующему его

```
interface Base {
    fun print()
}

class BaseImpl(val x: Int) : Base {
    override fun print() {
        print(x)
    }
}

class Derived(b: Base) : Base by b

fun main() {
    val b = BaseImpl(10)
    Derived(b).print()
}
```

typealias

- Не вводит новых типов, позволяет использовать альтернативные имена для существующих типов

- Для удобства - например, если существующее имя слишком длинно
- Для абстракции, чтоб легко менять тип

```
typealias NodeSet = Set<Network.Node>
typealias FileTable<K> = MutableMap<K, MutableList<File>>
typealias MyHandler = (Int, String, Any) -> Unit
typealias Predicate<T> = (T) -> Boolean
```

Неофициальное ДЗ – для тех, кто хочет потренироваться

- **hw1-KtTest.kt**
- Почините тесты в файле, используя элементы ООП, о которых мы сегодня говорили

Вопросы?

Рефлексия

Вопросы

1. Каковы основные концепции ООП?
2. Каковы основные принципы ООП?
3. Как устроена объектная модель Kotlin?

Тайминг: 2 мин

Пишем в чат

/

Говорим голосом

Тезисы

Подведем итоги

1. Программа в концепции ООП - совокупность взаимодействующих объектов, позволяет структурировать код
2. Основные элементы – интерфейс, класс, объект
3. Основные принципы: абстракция данных, инкапсуляция, наследование, полиморфизм
4. Важные возможности: ограничение доступа, обобщенное программирование

Цели вебинара

Проверка достижения целей:

1. Знаю элементы объектной модели Kotlin
2. Знаю, как создавать различные классы и объекты
3. Могу ответить на вопрос, как выполнять наследование и делегирование
4. Знаю, как пользоваться геттерами и сеттерами

Напишите в чат номера целей, которые вы достигли

Рефлексия

С какими впечатлениями уходите с вебинара?

Как будете применять на практике то,
что узнали на вебинаре?

Следующий вебинар

Модуль 2: Расширенные возможности Kotlin

Предметно ориентированные языки (DSL)

- Ссылка на вебинар будет в ЛК за 15 минут
- Материалы к занятию в ЛК — можно изучать
- Обязательный материал обозначен красной лентой

Список материалов для изучения

1. https://kotlinlang.org/
2. https://kotlinlang.org/docs/classes.html
3. https://kotlinlang.org/docs/properties.html
4. https://kotlinlang.org/docs/generics.html
5. https://kotlinlang.org/docs/visibility-modifiers.html
6. https://kotlinlang.org/docs/data-classes.html
7. https://kotlinlang.org/docs/inline-classes.html
8. https://kotlinlang.org/docs/enum-classes.html
9. https://kotlinlang.org/docs/object-declarations.html
10. https://quickbirdstudios.com/blog/kotlin-value-classes
11. https://github.com/otuskotlin/202505-ok-marketplace

Заполните, пожалуйста, опрос о занятии по ссылке в чате

Спасибо за внимание!
Приходите на следующие вебинары